Source Academy Chatbot Architecture

Overview

This document explains the two chatbot setups in the repository:

- **Louis**: the SICP page chatbot
- **Pixel**: the academy course RAG chatbot

Both use the same floating UI shell, but they differ in backend endpoints, context payload, and page integration.

---

1. Louis chatbot (SICP bot)

1.1 Entry point

- `src/pages/sicp/subcomponents/chatbot/Chatbot.tsx`
- The component wraps `FloatingChatbot` and renders `ChatBox`
- Intro text: `I am Louis, your SICP bot`

1.2 Floating UI shell

- `src/components/ui/chatbot/FloatingChatbot.tsx`
- Provides:
- draggable floating button
- open/close toggle
- expanded/shrunk state
- active code snippet state
- viewport clamping

1.3 Chat UI and behavior

- `src/pages/sicp/subcomponents/chatbot/ChatBox.tsx`
- Manages:
- `messages`
- `userInput`
- `isLoading`
- `maxContentSize`
- On mount it calls `resetChat()` to initialize the conversation.
- On send it calls `continueChat()` and appends the assistant response.
- A `clean` button resets the chat by calling `initChat()`.

1.4 Page context feeding

- `src/new_routes/sicpjs/_layout.tsx`
- Provides two callback props to `Chatbot`:
- `getSection()` returns the current SICP section from the URL path
- `getText()` scans visible paragraphs on the page and returns only text currently visible in the viewport
- This means Louis receives both:
- section identifier
- visible section text as `initialContext`

1.5 Backend API integration

- `src/features/sicp/chatCompletion/api.ts`
- `initChat(tokens)`

- POST to backend path `chats`
- body: `{}`

- `continueChat(tokens, userMessage, section, visibleText)`

- POST to backend path `chats/message`
- body includes:
- `message`
- `section`
- `initialContext`

- Uses `request()` from `src/commons/utils/RequestHelper.tsx`

- attaches `Authorization: Bearer `
- handles 401 refresh flow with `refreshToken`
- retries once after refresh

1.6 Message rendering

- `ChatBox.tsx` contains `MessageRenderer`
- It parses markdown-style code fences: ```` ```lang ... ``` ````
- For code blocks, it uses `ChatbotCodeSnippet`
- This enables clickable runnable snippets in Louis responses

1.7 Code snippet behavior

- `src/pages/sicp/subcomponents/chatbot/ChatbotCodeSnippet.tsx`
- Displays highlighted code and opens it in an overlay Playground when clicked
- It uses `WorkspaceActions.resetWorkspace('sicp')` and `WorkspaceActions.toggleUsingSubst(false, 'sicp')`
- This gives the user an interactive environment for code returned by Louis

1.8 Key limitation

- Louis only sends visible page text as context, not the whole section or book content
- That makes it more local to the currently visible SICP paragraphs

---

2. Pixel chatbot (academy RAG chatbot)

2.1 Entry point

- `src/pages/academy/ragChatbot/RagChatbot.tsx`
- It wraps `FloatingChatbot` and renders `RagChatBox`
- Intro text: `I am Pixel, your CS1101S assistant`

2.2 Floating UI shell

- Uses the same `FloatingChatbot.tsx`
- Supports the same floating behavior, drag, expand, collapse, and active snippet management

2.3 Chat UI and behavior

- `src/pages/academy/ragChatbot/RagChatBox.tsx`
- Manages:

- `messages`
- `userInput`
- `isLoading`
- `maxContentSize`

- On mount it calls `resetChat()` to initialize the RAG conversation
- On send it calls `sendRagMessage()` and appends the assistant response
- A `clean` button resets the chat by calling `initRagChat()`

2.4 Markdown and code rendering

- Uses `ReactMarkdown` + `remarkGfm`
- Markdown responses are rendered with native markdown structure
- Code blocks become clickable `ChatbotCodeSnippet` components
- Inline code stays inline

2.5 Backend API integration

- `src/features/ragChat/api.ts`
- `initRagChat(tokens)`

- POST to backend path `rag_chat`
- body: `{}`

- `sendRagMessage(tokens, userMessage)`

- POST to backend path `rag_chat/message`
- body includes:
- `message`

- Uses the same `request()` helper for auth and retry management

2.6 Page integration

- `src/new_routes/courses/[courseId]/_layout.tsx`
- `RagChatbot` is rendered inside academy course pages
- This makes Pixel available app-wide for academy course users

2.7 Key difference from Louis

- Pixel does not send page text or section context from the frontend
- It only sends the user prompt to the backend via `rag_chat/message`
- Louis sends extra page context and section metadata

2.8 Semantic purpose

- Pixel appears aimed at course-related help in academy pages
- Louis appears aimed at SICP content help within the SICP reader

---

3. Shared architecture

3.1 Shared floating shell

- Both chatbots use `src/components/ui/chatbot/FloatingChatbot.tsx`
- This is the shared UI container for floating chatbot widgets

3.2 Shared snippet renderer

- Both use `src/pages/sicp/subcomponents/chatbot/ChatbotCodeSnippet.tsx`

- It converts code blocks into runnable playground snippets

3.3 Shared request infrastructure

- Both use `src/commons/utils/RequestHelper.tsx`
- Shared logic:

- JSON body handling
- Authorization header generation
- 401 refresh token handling
- fallback error handling

3.4 Auth tokens

- Both chatbots use `useTokens` from `src/commons/utils/Hooks.ts`
- `useTokens()` reads `accessToken` and `refreshToken` from Redux session
- If tokens are missing or expired, the helper refreshes or logs out

---

4. Practical notes

4.1 Louis

- good for SICP-specific questions with visible paragraph context
- limitation: only currently visible paragraphs are sent as `initialContext`
- uses SICP section path plus visible text

4.2 Pixel

- good for academy course assistance general chat
- likely backed by RAG logic in the backend
- frontend does not send page content or section metadata

---

5. Files referenced

- `src/pages/sicp/subcomponents/chatbot/Chatbot.tsx`
- `src/pages/sicp/subcomponents/chatbot/ChatBox.tsx`
- `src/pages/sicp/subcomponents/chatbot/ChatbotCodeSnippet.tsx`
- `src/new_routes/sicpjs/_layout.tsx`
- `src/features/sicp/chatCompletion/api.ts`
- `src/pages/academy/ragChatbot/RagChatbot.tsx`
- `src/pages/academy/ragChatbot/RagChatBox.tsx`
- `src/features/ragChat/api.ts`
- `src/new_routes/courses/[courseId]/_layout.tsx`
- `src/components/ui/chatbot/FloatingChatbot.tsx`
- `src/commons/utils/RequestHelper.tsx`
- `src/commons/utils/Hooks.ts`